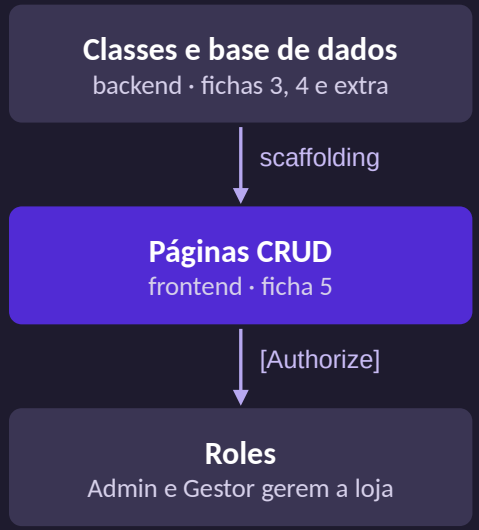
PROGRAMAÇÃO WEB · 2026/27 · AULA LABORATORIAL

Ficha 5
Páginas, CRUD e Roles

Antes, um extra ao modelo de dados. Depois, o Visual Studio gera as páginas de gestão a partir das classes, e nós damos-lhes o aspeto da loja, imagens, mensagens e regras de acesso por role.



PLANO DA AULA

Como lá vamos chegar

E

Extra: modelo de dados

Fora do enunciado. Morada, validações, nomes únicos, segunda migration e dados de exemplo.

1

Layout e páginas fixas

Alíneas 4.a a 4.d. Página inicial, Sobre, barra horizontal e menu, com o nome e as cores da loja.

2

Páginas CRUD geradas

Alíneas 4.e a 4.h. O scaffolding escreve as cinco páginas de cada entidade. Acerta-se o que não faz sozinho.

3

Customização e roles

Alíneas 4.i a 4.l. Português, imagens, mensagens, confirmação, caixas de escolha e acesso por role.

4

Acesso à aplicação

Etapa 5. Login como administrador; os dados de exemplo e as regras a funcionar.

5

Complemento

Um segundo utilizador, com o role Gestor: gere a loja, mas não as pessoas.

Como usar os materiais: estes slides explicam os conceitos, pela ordem em que são precisos na ficha. O guia HTML da ficha 5 tem o passo a passo, alínea a alínea, com o código completo para copiar e a indicação exata de onde fica cada ficheiro.

PONTO DE PARTIDA

O backend já existe, falta o frontend

Backend: já existe (fichas 3 e 4)

- As classes `Categoria`, `Produto`, `ModoEntrega` e `ApplicationUser`.
- O `ApplicationDbContext`, com três `DbSet`.
- A base de dados `FrutariaPWeb`, com as tabelas.
- Os roles `Admin`, `Gestor`, `Cliente` e o utilizador `admin@localhost.com`.
- As imagens da interface em `wwwroot/img`.

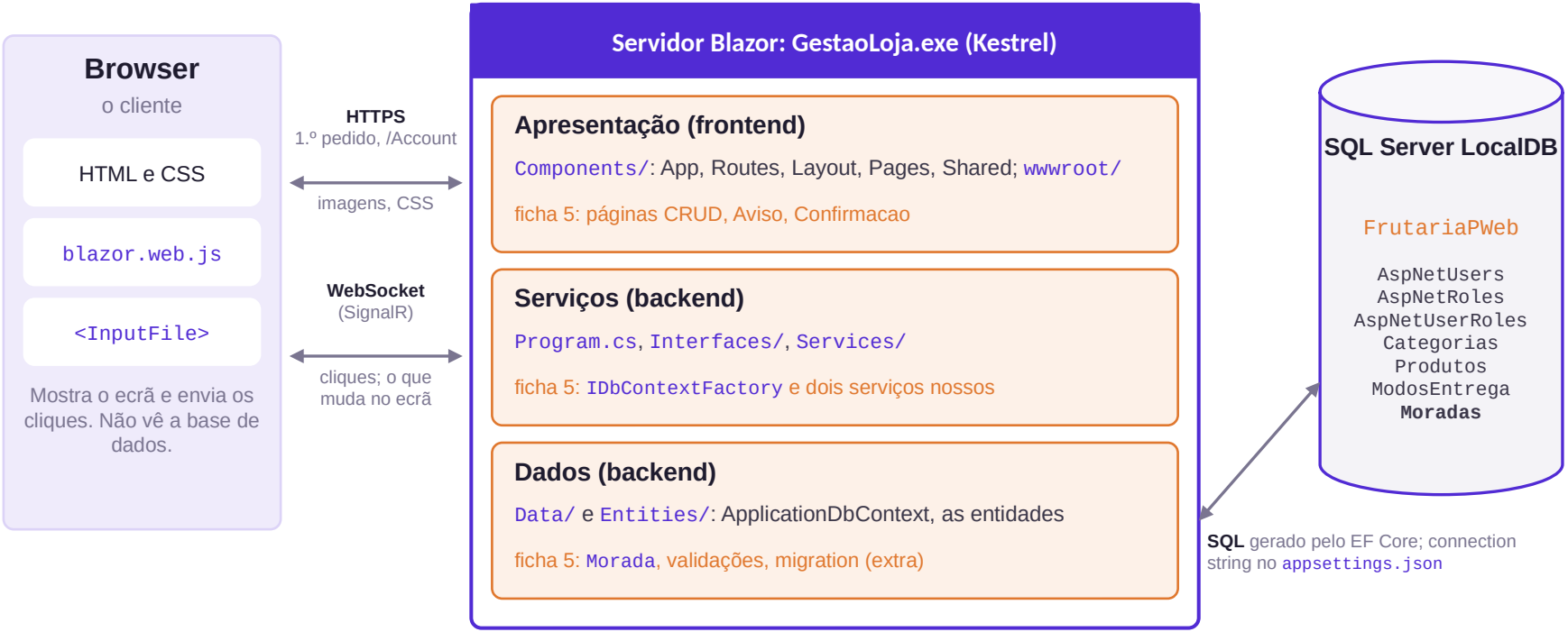
Frontend: falta (esta ficha)

- O aspeto da loja: barra horizontal, menu, cores.
- As páginas para gerir produtos, categorias e modos de entrega.
- Uma forma de escolher e guardar imagens.
- Mensagens de sucesso e de erro, e confirmação antes de apagar.
- Nas páginas, as regras: quem pode entrar.

Antes do frontend, um extra mexe no backend: a morada passa para uma tabela própria, as classes ganham validações, e a base de dados recebe uma segunda migration e dados de exemplo. Agora, enquanto ainda não tem dados.

CONTEXTO · ARQUITETURA

A aplicação a correr: três programas



CONTEXTO · CAMADAS

Dentro do servidor: três camadas

Apresentação (frontend)

`Components/`, `wwwroot/`

O que o utilizador vê e usa. Corre aqui, no servidor; o browser só recebe o HTML.

`App.razor`	`Routes.razor`	MainLayout, NavMenu, TopBar
Pages: Home, Sobre	Account: login, registo	wwwroot: img, css
CategoriaPages, ProdutoPages, ModoEntregaPages (CRUD)		Shared: Aviso, Confirmacao

Serviços (backend)

`Program.cs`, `Interfaces/`, `Services/`

Registados no arranque; as páginas pedem-nos com `@inject` (injeção de dependências).

Identity	AuthenticationState	`IDbContextFactory`	`IImagensService`, `IMensagensService`

Dados (backend)

`Data/` e `Entities/`

Classes C# que o EF Core traduz para tabelas e para SQL.

`ApplicationDbContext`	Categoria, Produto, ModoEntrega, ApplicationUser, **Morada** (extra)

Um clique em Gravar (criar categoria)

1. **Browser**: o `blazor.web.js` envia o clique pelo WebSocket.
2. **Apresentação**: o `Create.razor` corre `AddCategoria()`.
3. **Serviços**: a `IDbContextFactory`, recebida por `@inject`, cria um contexto.
4. **Dados**: `Add` marca, `SaveChangesAsync` gera o `INSERT`.
5. **LocalDB** grava (o `INSERT` vê-se na consola). A página volta à listagem, com a mensagem a verde; ao browser chega só o que mudou.

CONTEXTO · EXECUTAR

Ao carregar em F5: onde se vê cada parte

1. O browser

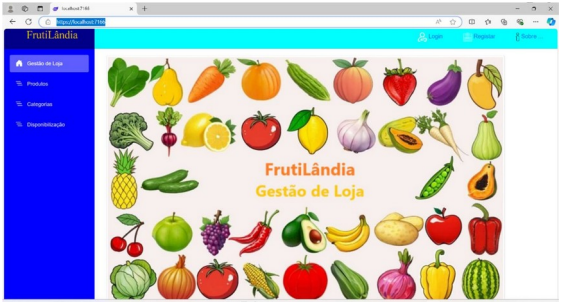


Figura 13 da ficha, **sem sessão iniciada**. Abre sozinho em `https://localhost:7166` (a porta muda de projeto para projeto).

2. A consola do servidor

```
info: Microsoft.Hosting.Lifetime
  Now listening on:
https://localhost:7166
info: Microsoft.EntityFrameworkCore
      .Database.Command
  Executed DbCommand (3ms)
  SELECT [c].[Id], [c].[Nome] ...
  FROM [Categorias] AS [c]
```

É o `GestaoLoja.exe`: os logs do servidor, com o SQL do EF Core. A consola do browser (F12) é outra.

3. A base de dados

Pesquisador de Objetos do SQL Server

```
(localdb)\MSSQLLocalDB
  Bancos de Dados
    FrutariaPWeb
      Tabelas
        dbo.Produtos
```

Outro processo, que arranca sozinho. Exibir Dados mostra o conteúdo das tabelas.

Shift+F5 termina o `GestaoLoja.exe`: o browser fica sem servidor. O LocalDB continua a correr, e os dados ficam.

EXTRA · FORA DO ENUNCIADO

Antes da alínea 4.a: um extra ao modelo de dados

Porquê agora

1. A base de dados ainda só tem os roles e o administrador: **mudar tabelas não custa nada**.
2. O scaffolding (4.e) lê as classes: as páginas **já saem com as regras novas**.
3. Vê-se uma **segunda migration**: a primeira criou tabelas, esta altera-as.

O resto da ficha parte do princípio de que o extra foi feito.

Ficheiro	O que muda	Na base de dados
`Morada.cs` (novo)	A morada passa a classe própria	Tabela `Moradas`
`ApplicationUser.cs`	Sai a morada; entra `MoradaId`	`AspNetUsers` alterada
`Categoria.cs`, `Produto.cs`	Validações; sai o `ImageFile`	`Nome` passa a `nvarchar(100)`
`ApplicationDbContext.cs`	Nomes únicos; regras `CHECK`	Índices e restrições
`Inicializacao.cs`	Dados de exemplo; admin `Activo`	Só dados
`Program.cs`	Confirma as migrations; dados de exemplo; registo sem confirmar o email	Nada

EXTRA · MORADA

A morada numa tabela própria

Antes (fichas 3 e 4)

AspNetUsers
Id, Email, ...
Nome, Apelido
Rua
Localidade
Cidade
CodigoPostal
Pais
NIF, Telemovel, ...

Depois (extra)

AspNetUsers
Id, Email, ...
Nome, Apelido
MoradaId (FK, NULL)
NIF, Telemovel, ...

Moradas
Id (PK)
Rua, Localidade
Cidade, CodigoPostal
Pais

Mesmo padrão de `CategoriaId` e `categoria` no `Produto` (ficha 3): o EF Core descobre a relação pelos nomes.

Data/ApplicationUser.cs (as linhas novas)

```
public int? MoradaId { get; set; }
public Morada? morada { get; set; }
// MoradaId: a coluna; morada: o objeto
```

Porquê numa tabela

`AspNetUsers` é a tabela das **contas**. A morada vai servir também as **encomendas** (morada de entrega): a mesma classe, as mesmas regras.

Porquê int?

Um utilizador pode não ter morada: quem se regista só dá o email e a palavra-passe. Com `int`, o registo falhava.

CONCEITO · VALIDAÇÃO

Regras em duas camadas: a classe e a base de dados

1. Na classe (`Entities/Produto.cs`): mensagens claras

```
[Range(0.01, 99999999.99, ErrorMessage = "...")]
public decimal Preco { get; set; }

// regra com dois campos: IValidatableObject
public IEnumerable<ValidationResult> Validate(...)
{  if (Disponivel && EmStock <= 0)
     yield return new ValidationResult("...", ...);  }
```

2. Na base de dados (`OnModelCreating`): a garantia

```
// nomes únicos: CREATE UNIQUE INDEX
builder.Entity<Categoria>().HasIndex(c => c.Nome)
       .IsUnique();

// condições: CHECK
builder.Entity<Produto>().ToTable(t => t.HasCheckConstraint(
    "CK_Produtos_Preco", "[Preco] > 0"));
```

Quem grava	Quem verifica a classe	Se a regra falhar
Formulário (`EditForm`)	`DataAnnotationsValidator`	Mensagem ao lado do campo
API (mais tarde)	O ASP.NET Core, sozinho	Responde `400 Bad Request`
Outro código C#	**ninguém**	Só a base de dados o trava

O EF Core não valida

`SaveChangesAsync` não olha para `[Range]` nem para `Validate`. Daí a 2.ª camada: recusa com `DbUpdateException`.

EXTRA · MIGRATIONS

A primeira migration cria; as seguintes alteram

Console do Gerenciador de Pacotes, um de cada vez

```
Add-Migration MoradaEValidacoes
Update-Database
Script-Migration -From 0 -To BDInicial
Script-Migration -From BDInicial -To MoradaEValidacoes
```

"may result in the loss of data"

Aviso do `Add-Migration`: apaga as colunas da morada e encurta `Nome`. Aqui não há dados. **Com dados, parar e ler a migration.**

BDInicial (ficha 4): CRIA

```
CREATE TABLE [Categorias] (
  [Id] int NOT NULL IDENTITY,
  [Nome] nvarchar(max) NULL,
  [Ordem] int NOT NULL,
  [UrlImagem] nvarchar(max) NULL,
  [Imagem] varbinary(max) NULL,
  CONSTRAINT [PK_Categorias] ...
);
```

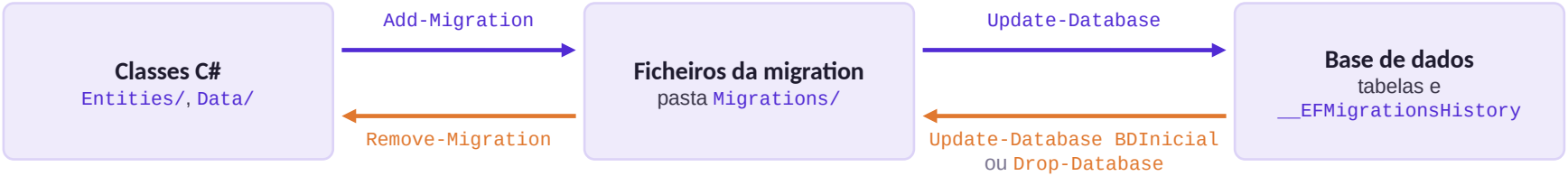
MoradaEValidacoes (extra): ALTERA

```
ALTER TABLE [AspNetUsers] DROP COLUMN [Rua];
ALTER TABLE [AspNetUsers] ADD [MoradaId] int NULL;
ALTER TABLE [Categorias] ALTER COLUMN [Nome]
    nvarchar(100) NOT NULL;
CREATE TABLE [Moradas] (...);   -- a única nova
CREATE UNIQUE INDEX [IX_Categorias_Nome] ...
ALTER TABLE [Produtos] ADD CONSTRAINT
    [CK_Produtos_Preco] CHECK ([Preco] > 0);
```

Cada migration envia **só a diferença** entre a fotografia anterior e as classes. Esquecer o `Update-Database`: a aplicação não arranca e diz o que falta.

EXTRA · MIGRATIONS

Enganei-me numa migration: como voltar atrás



Porquê esta ordem? O `Update-Database` regista a migration na tabela `__EFMigrationsHistory`. Enquanto lá estiver, o `Remove-Migration` recusa apagá-la, e voltar a correr o `Update-Database` não a repete. **Desfaz-se primeiro na base de dados, depois nos ficheiros.**

A situação	Primeiro	Porquê
Ainda **não** fiz `Update-Database`	`Remove-Migration`	A base de dados não sabe desta migration: basta apagar os ficheiros.
Já fiz `Update-Database` e quero manter os dados	`Update-Database BDInicial`, depois `Remove-Migration`	Volta uma migration atrás: corre o `Down` dela e tira-a do histórico.
Já fiz `Update-Database`, sem dados a perder (ficha 4)	`Drop-Database`, depois `Remove-Migration`	Apaga a base de dados, e o histórico com ela. No F5, a `Inicializacao` repõe os dados.
A migration já foi para o git do grupo	**Não se desfaz**	Os colegas já a aplicaram: corrige-se com uma migration nova.

Nas três primeiras, a seguir: corrigir as classes, `Add-Migration` e `Update-Database`. Nunca apagar os ficheiros à mão.

EXTRA · BASE DE DADOS

Onde estão os ficheiros da base de dados

```
Na pasta do utilizador do Windows (não na pasta do projeto)
C:\Users\<utilizador>\FrutariaPWeb.mdf       os dados
C:\Users\<utilizador>\FrutariaPWeb_log.ldf   o registo
```

```
Confirmar: clique direito em FrutariaPWeb > Nova Consulta...
SELECT name, physical_name FROM sys.database_files;
```



A aplicação nunca abre os ficheiros: fala com o LocalDB, que os tem abertos.

A connection string tem o nome

`Server=(localdb)\MSSQLLocalDB;` `Database=FrutariaPWeb`: o nome da base de dados, não o caminho. Cada computador tem a sua.

Não se mexe à mão

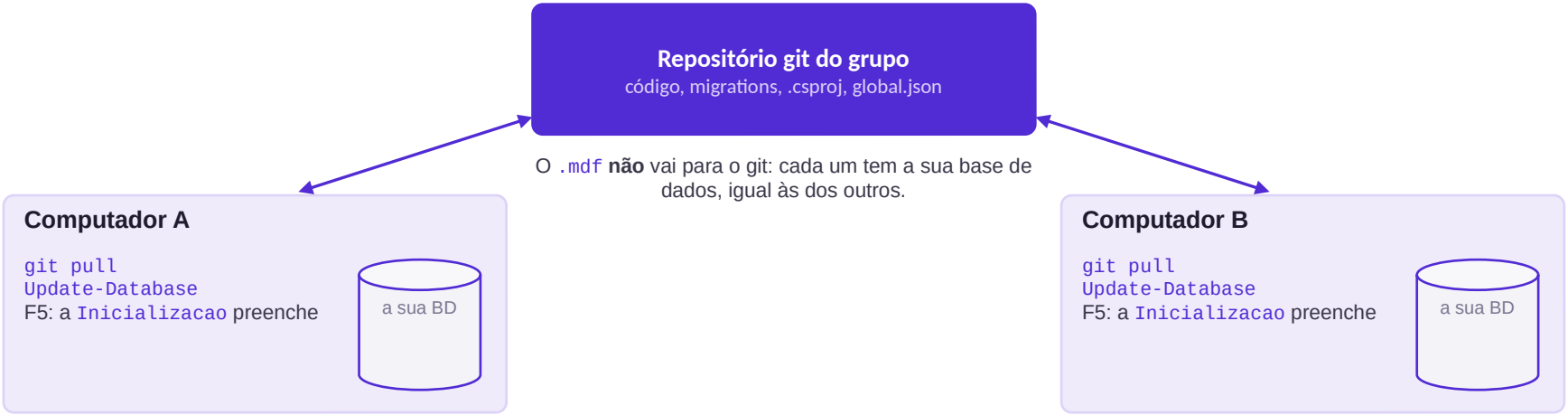
Copiar, mover ou apagar o `.mdf` com o Visual Studio aberto estraga a base de dados. Para apagar: `Drop-Database`.

Ver as tabelas

Pesquisador de Objetos do SQL Server, como na ficha 4. Exibir Dados mostra as linhas.

EXTRA · TRABALHO EM GRUPO

Em grupo: o código partilha-se, a BD recria-se



Regra	Como
Onde	`C:\PWeb\GestaoLoja`, na raiz do disco. **Fora do OneDrive** e do Ambiente de Trabalho (bloqueia ficheiros, caminhos longos)
git	`dotnet new gitignore` na pasta da solução, antes do 1.º commit: deixa de fora `bin/`, `obj/`, `.vs/`
Migrations	Uma de cada vez, combinada no grupo. Nunca alterar uma que já foi para o git. Esquecer o `Update-Database`: a app diz quais faltam
Versões	O mesmo Visual Studio; `global.json` fixa o SDK; pacotes NuGet só se atualizam com o grupo todo

EXTRA · DADOS DE EXEMPLO

A loja arranca já com dados, e com imagens

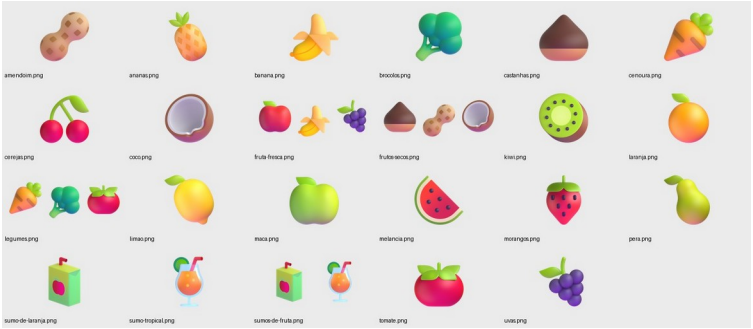
Data/Inicializacao.cs: CriaDadosExemplo (resumido)

```
if (await context.Categorias.AnyAsync()) return;
// já há dados: não duplica
var fruta = new Categoria { Nome = "Fruta Fresca",
    Imagem = Le("fruta-fresca.png") };
context.Produtos.AddRange(P("Laranja", ..., fruta, kg), ...);
await context.SaveChangesAsync();
// um só, no fim: todos os INSERT
```

Program.cs, no arranque (resumido)

```
// 1. faltam migrations? pára, com uma mensagem clara
var pendentes = await context.Database.GetPendingMigrationsAsync();
if (pendentes.Any()) throw new InvalidOperationException(...);
// 2. roles e administrador; 3. dados de exemplo
await Inicializacao.CriaDadosExemplo(context, ...seed);
```

Só da primeira vez: com categorias já gravadas, não faz nada. 4 modos, 4 categorias, 19 produtos.



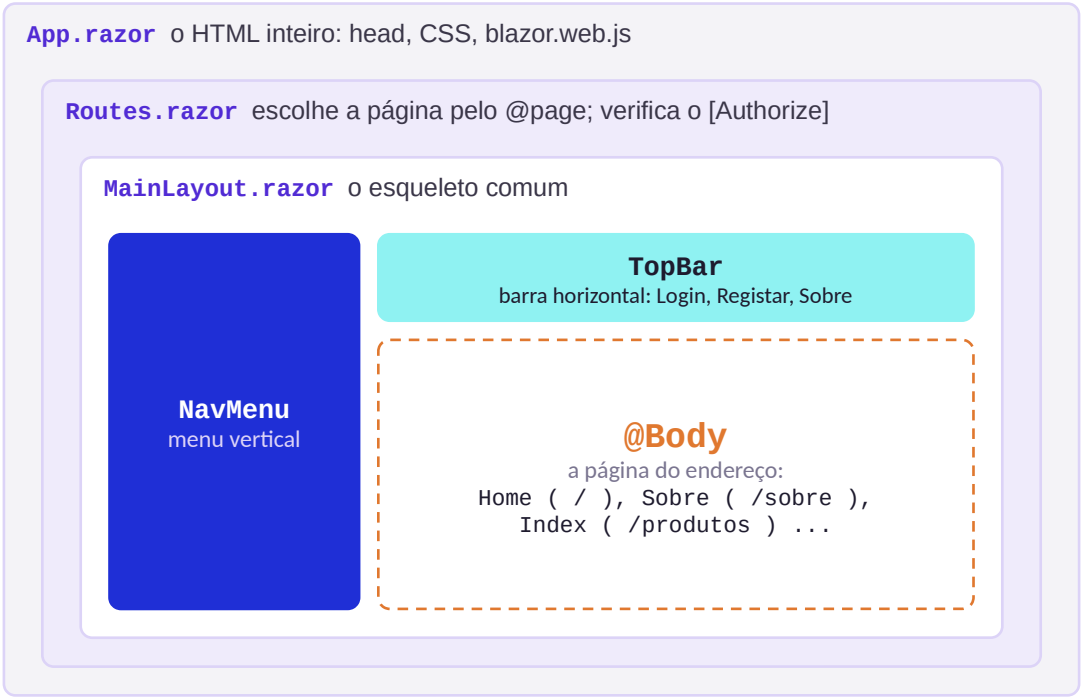
`wwwroot/img/seed/`: as 23 imagens (`Ficha5_imagens_exemplo.zip`)

O formato das imagens da loja

PNG com fundo transparente, **220 x 120** píxeis, idealmente **menos de 100 KB** (máximo **500 KB**). Cada listagem envia todas as imagens ao browser, em base64: uma fotografia de 3 MB pesa 4 MB por cada vez que a página abre.

ALÍNEAS 4.B A 4.D · LAYOUT

Como uma página chega ao ecrã



@page faz a página

Um componente com `@page "/sobre"` tem endereço. Sem `@page`, só pode ser usado dentro de outro (como o `NavMenu`).

Tudo veio do template

App, Routes, MainLayout e NavMenu foram criados na ficha 3, com Contas Individuais. O `TopBar` é novo.

Nesta ficha

Alteramos `Home`, `MainLayout` e `NavMenu`; criamos `Sobre` e `TopBar`. O `Routes` muda mais tarde, por causa dos roles.

ALÍNEAS 4.B E 4.C · PÁGINAS FIXAS

Home e Sobre: o @page, o AppConfig e o wwwroot

Components/Pages/Sobre.razor (resumido)

```
@page "/sobre"

<PageTitle>Sobre</PageTitle>
<h1>Sobre nós</h1>
<p><b>@AppConfig.tituloHomePage</b></p>
<p>@AppConfig.enderecoHome</p>
<p>Email: @AppConfig.mail</p>
```

O endereço

`@page "/"` é a Home; `@page "/sobre"` é o Sobre. É o `Router` que liga um ao outro.

Os textos vêm do AppConfig

Classe `static` da ficha 3, no namespace `GestaoLoja`: visível em todas as páginas. Muda-se a morada num só sítio.

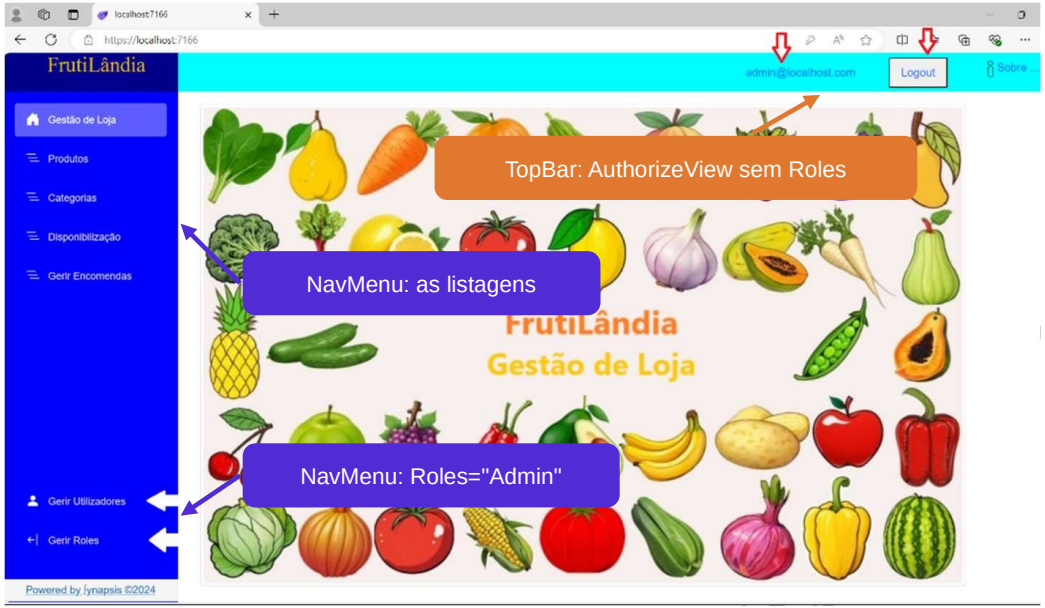
A imagem: wwwroot e Home.razor.css

`wwwroot/img/frutaria.jpg` fica em `/img/frutaria.jpg` (`UseStaticFiles`). `width: 100%`, `max-height` e `object-fit` fazem-na acompanhar a janela.

Minúsculas

No Windows `Frutaria.jpg` e `frutaria.jpg` são o mesmo ficheiro; num servidor Linux não. Escrever como está na pasta.

O que mudou de sítio no layout



Login e Logout mudaram de sítio

No template estavam no `NavMenu`. Agora estão na barra horizontal, num componente novo, o `TopBar`, com o código que os acompanha (`currentUrl`). Cada componente com uma função.

O menu é da loja

Nome da loja (`AppConfig`), Gestão de Loja, Produtos, Categorias, Disponibilização e, em baixo, as entradas do Admin.

Logout é um POST

Termina a sessão (a resposta manda o browser apagar o cookie): formulário com `<AntiforgeryToken />` para `Account/Logout`.

ALÍNEA 4.D · COMPONENTES

Um componente: diretivas, HTML, C# e CSS

Components/Layout/TopBar.razor (resumido)

```
@implements IDisposable
@inject NavigationManager NavigationManager

<AuthorizeView>
  <Authorized> email e Logout </Authorized>
  <NotAuthorized> Login, Registar </NotAuthorized>
</AuthorizeView>
<div class="nav-login"> Sobre </div>

@code {
  private string? currentUrl;
  protected override void OnInitialized() =>
    NavigationManager.LocationChanged += OnLocationChanged;
  public void Dispose() =>
    NavigationManager.LocationChanged -= OnLocationChanged;
}
```

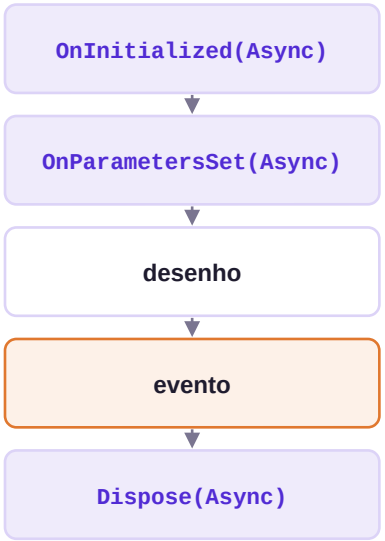
Zona	O que lá está
Diretivas	`@implements`, `@inject`, `@page`, `@using`: o que o componente é e de que precisa
Marcação	HTML (`<div>`) e componentes (tags com maiúscula: `<AuthorizeView>`, `<NavLink>`), com C# depois de `@`
`@code`	C# normal: campos, métodos, ciclo de vida (`OnInitialized`)
CSS	`TopBar.razor.css`, ao lado: só se aplica a este componente

Isolamento de CSS

Todos os `.razor.css` são juntos em `GestaoLoja.styles.css`. `::deep` chega aos elementos dos componentes filhos (os `<a>` do `NavLink`). O ciclo de vida e o `+=` estão no slide seguinte.

CONCEITO · CICLO DE VIDA

Ciclo de vida de um componente, e eventos



Uma vez, quando o componente é criado, antes do 1.º desenho

Sempre que recebe parâmetros novos

O Blazor gera o HTML e envia ao browser o que mudou

Um clique, ou `StateHasChanged()`: volta a desenhar

Quando sai do ecrã (só com `@implements IDisposable`)

Na ficha: `TopBar` (`OnInitialized`, `LocationChanged`, `Dispose`); listagens (`OnInitialized` cria o contexto, `DisposeAsync` liberta-o); `Edit`, `Details`, `Delete` (`OnInitializedAsync` lê o registo); `Aviso` (`OnInitialized` e `OnParametersSet`).

Eventos da interface

`@onclick`, `OnValidSubmit`, `OnChange`: vêm do browser, pela ligação SignalR. No fim do método, o Blazor desenha de novo sozinho.

Eventos C#, de um serviço

`NavigationManager.LocationChanged`: `+=` subscreve ("avisa-me"), `-=` cancela. Não vem da interface, por isso o Blazor não sabe que algo mudou: `StateHasChanged()`. Cada `+=` precisa de um `-=` no `Dispose`.

Pré-renderização

Com F5 ou um endereço escrito, `OnInitialized` corre duas vezes: no HTML estático e quando a ligação SignalR se estabelece.

ALÍNEA 4.D · AUTHORIZEVIEW

AuthorizeView com e sem Roles

Onde	Escrito como	Pergunta	Porquê assim
Barra horizontal (`TopBar`)	`<AuthorizeView>`	Há sessão iniciada? (autenticação)	Quem entrou vê o email e o Logout, seja qual for o role
Menu: Gerir Encomendas	`<AuthorizeView Roles="Admin,Gestor">`	Tem um destes roles? (autorização)	É trabalho de quem gere a loja, não de um cliente
Menu: Utilizadores, Roles	`<AuthorizeView Roles="Admin">`	É Admin?	Dar e tirar permissões é o mais sensível
Páginas CRUD	nenhum: só `[Authorize]`	(quem entra pode tudo)	Admin e Gestor gerem a loja: não há nada a esconder lá dentro

Sem Roles

Pergunta só **quem és** (autenticação). Basta ter feito login.

Com Roles

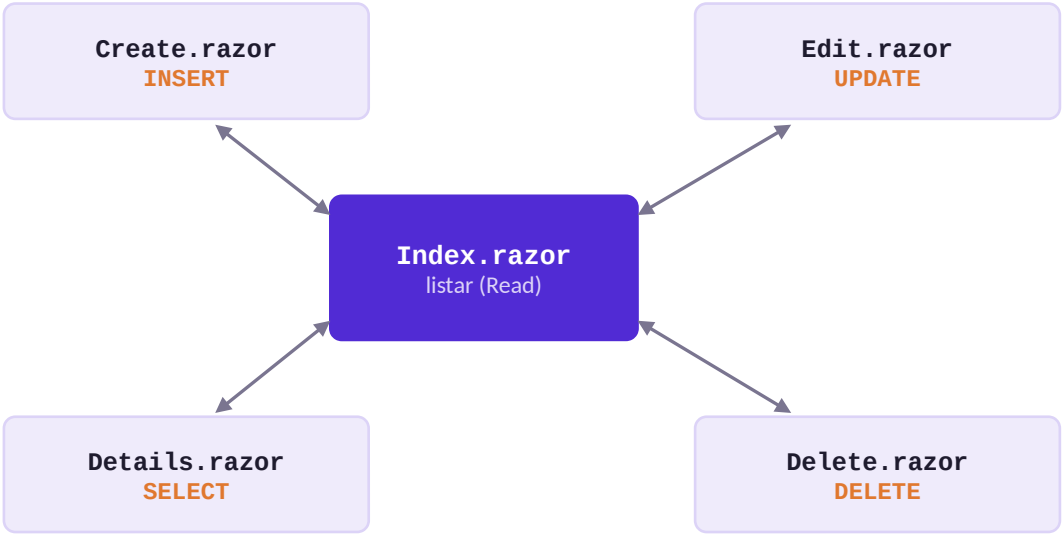
Pergunta **o que podes** (autorização). A vírgula é um OU. Os nomes são os da tabela `AspNetRoles`.

Esconder não é proteger

O `AuthorizeView` só decide o que se desenha. Quem bloqueia a página é o `[Authorize]` (alíneas 4.i a 4.k).

CONCEITO · CRUD

CRUD: quatro operações, cinco páginas



Endereços: `/produtos`, `/produtos/create`, `/produtos/edit?id=3`, `/produtos/details?id=3`, `/produtos/delete?id=3`

Operação	No código (EF Core)
Create	`Add(obj)` + `SaveChangesAsync()`
Read	`QuickGrid` / `FirstOrDefaultAsync`
Update	`Attach(obj).State = Modified` + `SaveChangesAsync()`
Delete	`Remove(obj)` + `SaveChangesAsync()`

Só o SaveChanges escreve

`Add`, `Attach` e `Remove` só marcam o objeto. O SQL é gerado e enviado no `SaveChangesAsync`.

ALÍNEAS 4.E A 4.G · SCAFFOLDING

Scaffolding: o Visual Studio escreve as páginas

Entra (o assistente pede)

Campo	Valor
Modelo	`CRUD`
Classe de modelo	`Categoria (GestaoLoja.Entities)`
Classe DbContext	`ApplicationDbContext`, da lista
Provedor	SQL Server

Caminho: clique direito em `Components/Pages` > Adicionar > Novo Item em Scaffolding > Componentes Razor usando o Entity Framework (CRUD)

Sai (o Visual Studio cria e altera)

```
Components/Pages/CategoriaPages/
  Index.razor    /categoria
  Create.razor   /categoria/create
  Edit.razor     /categoria/edit?id=
  Details.razor  /categoria/details?id=
  Delete.razor   /categoria/delete?id=
```

`/categoria` no singular; `/produtos` e `/modoentregas` no plural (regras do inglês). + pacotes NuGet e linhas no `Program.cs`

Na Classe DbContext, escolher da lista. Não carregar no +

O + cria outro contexto e outra connection string, para uma base de dados que nem existe: as páginas deixavam de usar a `FrutariaPWeb`.

ALÍNEAS 4.E A 4.G · OS CAMPOS

Um campo por coluna: como o scaffolding escolhe

Tipo da propriedade	Componente gerado	Nas nossas classes
`string`	`InputText`	Nome, Detalhe, UrlImagem
`int`, `decimal`	`InputNumber`	Ordem, Preco, CategoriaId
`bool`	`InputCheckbox`	Promocao, Disponivel
`DateTime`	`InputDate`	(nenhuma)
qualquer outro	`InputText`	**`Imagem`** (**`byte[]`**)

Sem componente para bytes

Para `byte[]` não há input. O scaffolding cai na regra "outro" e gera `<InputText @bind-Value="Categoria.Imagem" />`: uma caixa de texto ligada a bytes. Não compila.

Só entra o que é coluna

O scaffolding pergunta ao EF Core que propriedades estão mapeadas. Ficam de fora a chave `Id` (gerada pela base de dados), as navegações (`categoria`, `produtos`) e o que tiver `[NotMapped]`.

Chaves estrangeiras

`CategoriaId` sai como `InputNumber`: escreve-se o número. Trocamos por uma caixa de escolha (4.j).

Regras confirmadas no código-fonte dos templates do .NET 8.

ALÍNEAS 4.E A 4.H · IMAGEM

Imagem (byte[]): a propriedade que parte o scaffold

CategoriaPages/Create.razor, tal como é gerado (excerto)

```
<div class="mb-3">
  <InputText @bind-Value="Categoria.Imagem" />
  <ValidationMessage For="() => Categoria.Imagem" />
</div>
```

Lista de Erros

```
CS1503 não é possível converter de 'byte[]' para 'string'
```

Porque é que não pode ficar para depois

O scaffolding **compila o projeto antes de gerar**. Com este erro por corrigir, o scaffolding da entidade seguinte falha. Acontece no `Create` e no `Edit` de `Categoria` e de `Produto`.

Seguido no guia

Gerar e, logo a seguir, apagar os dois blocos da `Imagem` (no `Create` e no `Edit`). Não se mexe nas classes, e as páginas vão ser reescritas.

Alternativa: comentar

Comentar `Imagem` antes e descomentar depois. Risco: esquecer, e o próximo `Add-Migration` gera `DropColumn("Imagem")`.

E o ImageFile?

Saiu no extra. Com `[NotMapped]`, também não dava problema.

SÓ PARA MAC · ALÍNEAS 4.E A 4.G

O mesmo scaffolding, pela linha de comandos

Com Windows e Visual Studio, usa-se o assistente (slide anterior) e salta-se este slide. Isto é para quem tem Mac, ou vê o assistente falhar.

Na pasta do projeto (a do GestaoLoja.csproj; no Terminal do Visual Studio: cd GestaoLoja)

```
# uma vez por computador: a ferramenta, na versão do .NET 8
dotnet tool install --global dotnet-aspnet-codegenerator --version 8.0.7
# uma vez por projeto: os pacotes de que o scaffolding precisa
dotnet add package Microsoft.VisualStudio.Web.CodeGeneration.Design --version 8.0.7
dotnet add package Microsoft.AspNetCore.Components.QuickGrid.EntityFrameworkAdapter --version 8.0.31  # = .csproj
# uma entidade de cada vez: gerar, apagar os blocos da Imagem, compilar; só depois a seguinte
dotnet aspnet-codegenerator blazor CRUD -dbProvider sqlserver -dc GestaoLoja.Data.ApplicationDbContext -m Categoria
dotnet build   # zero erros antes de gerar o Produto
```

Parte	Significado
`blazor CRUD`	O gerador Blazor, as cinco páginas
`-dbProvider sqlserver`	O motor de base de dados
`-dc ...ApplicationDbContext`	O contexto que já existe (com namespace)
`-m Categoria`	A classe de modelo

Onde ficam as páginas

Sem opção nenhuma: vão sempre para `Components/Pages/CategoriaPages`. Cada comando começa por compilar: correr os três de seguida dá "Build Failed" no segundo.

CONCEITO · INJEÇÃO DE DEPENDÊNCIAS

Interface, classe e tempo de vida

Program.cs: regista (interface, classe)

```
builder.Services.AddSingleton<IImagensService, ImagensService>();
builder.Services.AddScoped<IMensagensService, MensagensService>();
builder.Services.AddDbContextFactory<ApplicationDbContext>(...);
```

o contentor entrega o objeto

Uma página: pede a interface

```
@inject IImagensService Imagens
@inject IDbContextFactory<ApplicationDbContext> DbFactory
```

Registo	Quantos objetos	Aqui
`AddSingleton`	Um só, para a aplicação inteira	`ImagensService` (sem estado)
`AddScoped`	Um por âmbito (scope)	`MensagensService`, `UserManager`
`AddTransient`	Um novo de cada vez que é pedido	(nenhum)

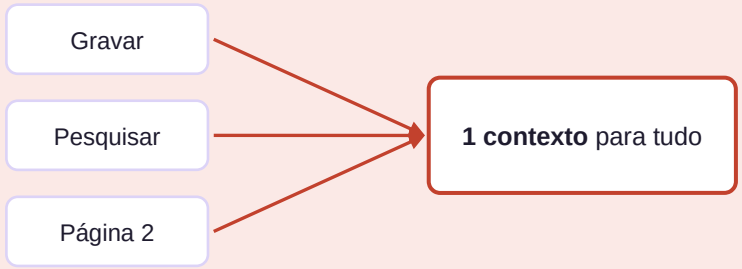
O âmbito (Scoped) é...	Numa API (ou página estática): um pedido HTTP	Numa página Blazor interativa: a ligação SignalR
Dura	Milissegundos	Enquanto o separador estiver aberto (horas)
DbContext	`AddDbContext`; o controlador pede `ApplicationDbContext`	`AddDbContextFactory`; a página pede `IDbContextFactory<...>`

CONCEITO · FÁBRICA DE CONTEXTOS

Porquê uma fábrica de contextos?

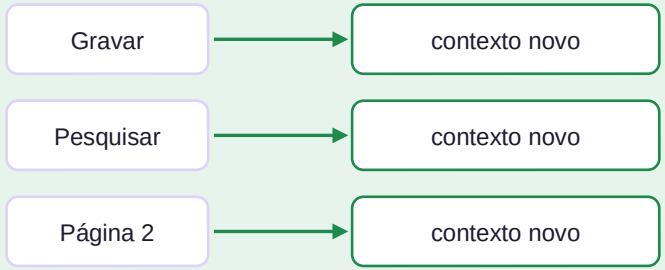
O `DbContext` é como um carrinho de compras: serve para **uma** ida às compras e é de **uma** pessoa de cada vez. A fábrica é a fila de carrinhos à entrada: cada operação tira um carrinho novo e larga-o no fim.

Sem fábrica: um contexto para a página toda



⚠️ Dois cliques ao mesmo tempo: **erro**.
⚠️ Guarda o que já leu: **dados antigos**.

Com fábrica: um contexto por operação



✓ Cada operação usa o seu e **larga-o no fim**.
✓ Dados sempre **acabados de ler**.

Porquê no Blazor: a página fica aberta (ligação SignalR) e não há "fim do pedido" para largar o contexto. É o `using` que o larga, no fim de cada método.

ALÍNEA 4.H · PROGRAM.CS

IDbContextFactory: um contexto por operação

O padrão das páginas geradas

```
@inject IDbContextFactory<ApplicationDbContext> DbFactory

using var context = DbFactory.CreateDbContext();
context.Categorias.Add(Categoria);
await context.SaveChangesAsync();
// no fim do método, o using liberta o contexto
```

Em resumo

Um carrinho por operação: `CreateDbContext()` tira um contexto novo, o `using` larga-o no fim do método. Assim nunca há duas operações no mesmo contexto, nem dados guardados de cliques anteriores.

O que o scaffolding não faz

Com um contexto já existente, não regista a fábrica. A primeira página dá: `There is no registered service of type IDbContextFactory`.

Program.cs: AddDbContext passa a AddDbContextFactory

```
builder.Services.AddDbContextFactory<
    ApplicationDbContext>(options =>
        options.UseSqlServer(connectionString));
```

Regista as duas coisas

A fábrica, para as páginas, e o `ApplicationDbContext` (Scoped), que o Identity continua a usar.

CONCEITO · MODOS DE RENDERIZAÇÃO

Páginas estáticas e páginas interativas

Components/App.razor (do template, interatividade Global)

```
RenderModeForPage => HttpContext.Request.Path.StartsWithSegments("/Account")
    ? null                // /Account: estáticas
    : InteractiveServer;  // todas as outras: interativas
```

	Estática (login, registo)	Interativa (as nossas páginas)
Carregar em Gravar	O browser faz um POST e recebe uma página nova	O clique vai pela ligação SignalR; o método corre no servidor; muda só o que mudou
Um ficheiro chega como	`IFormFile`, dentro do POST	`IBrowserFile`, pelo `<InputFile>`
`method="post"`, `FormName`, ...	São usados	Ficam no código gerado, sem efeito

Foi na ficha 3, Figura 3: **Tipo de interatividade: Servidor** e **Interatividade local: Global**. É por isso que as nossas páginas são interativas, e que o `IFormFile` não serve para as imagens: aqui chegava sempre vazio, e o registo era gravado sem imagem, sem dar erro.

CONCEITO · IMAGENS

IFormFile e IBrowserFile: dois envelopes

	IFormFile	IBrowserFile
Namespace	`Microsoft.AspNetCore.Http`	`Microsoft.AspNetCore.Components.Forms`
Como viaja	Dentro de um POST de formulário (multipart)	Pela ligação SignalR, enviado pelo `<InputFile>`
Onde funciona	Páginas estáticas (`/Account`), MVC, **API**	Páginas Blazor **interativas** (as nossas)
Nome, tamanho	`FileName`, `Length`	`Name`, `Size`
Ler o conteúdo	`OpenReadStream()`	`OpenReadStream(tamanhoMáximo)`

Os dois dão um **`Stream`** (os bytes, lidos do princípio ao fim). Por isso o serviço de imagens recebe um `Stream`: serve aqui e na API.

Ficam duas propriedades

`Imagem` (`byte[]`): os bytes. `UrlImagem`: o nome do ficheiro.

Porque saiu o ImageFile (extra)

Era um `IFormFile`: aqui chegava sempre `null`. E a entidade é uma linha da tabela, não o pedido.

COMPLEMENTO · SERVIÇO DE IMAGENS

IImagensService: as imagens num serviço

Interfaces/IImagensService.cs (o contrato)

```
public interface IImagensService
{  long TamanhoMaximo { get; }
   string? Valida(string nome, long tamanho);
   Task<byte[]> LerBytesAsync(Stream conteudo);
   string Src(byte[]? imagem, string? nome);  }
```

CategoriaPages/Create.razor (excerto)

```
@inject IImagensService Imagens
<InputFile OnChange="CarregaImagem" />

erroImagem = Imagens.Valida(e.File.Name, e.File.Size);
await using var conteudo = e.File.OpenReadStream(...);
Categoria.Imagem = await Imagens.LerBytesAsync(conteudo);
Categoria.UrlImagem = e.File.Name;
```

Porquê um serviço, e não uma classe static

A página pede a **interface**; o `Program.cs` escolhe a classe. Guardar as imagens em ficheiros, um dia, é escrever outra classe: as dez páginas ficam iguais. E é o padrão da API.

Singleton

Só tem funções e não guarda nada de ninguém: um objeto chega para a aplicação inteira.

Ainda não grava

O `OnChange` só preenche o objeto: a pré-visualização muda logo (`Imagens.Src`). O `INSERT` é no Gravar, já com os bytes.

COMPLEMENTO · FEEDBACK

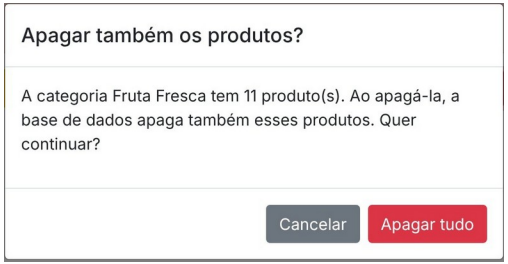
Mensagens a verde e a vermelho, e a confirmação

Categoria apagada, com os seus 11 produto(s). ×

Já existe uma categoria com o nome "Fruta Fresca". ×

`<Aviso />` (`Components/Shared`): Bootstrap `alert-success` / `alert-danger`

1. O `Create` grava e chama `Mensagens.Sucesso("Categoria criada.")`.
2. `NavigateTo("/categoria")`: muda de página **na mesma ligação SignalR**.
3. O `<Aviso />` da listagem, no `OnInitialized`, chama `Retira()`.
4. Mostra a verde; o serviço esquece-a (F5 já não a mostra).



`<Confirmacao>`
Modal do Bootstrap, desenhado pelo Blazor com `@if (Visivel)`. Sem JavaScript.

Porque é que IMensagensService é Scoped

A mensagem é **de um utilizador**: um objeto por separador (a ligação SignalR). Numa classe `static`, a mensagem de um aparecia no ecrã de outro. Os erros da própria página (nome repetido) vão pelo parâmetro: `<Aviso Erro="@erro" />`.

ALÍNEAS 4.I A 4.K · QUICKGRID

QuickGrid: o que se escreve e o HTML que sai

O que se escreve (CategoriaPages/Index.razor)

```
<QuickGrid Class="table" Items="context.Categorias">
  <TemplateColumn Context="categoria">
    <img src="@Imagens.Src(...)" />
  </TemplateColumn>
  <PropertyColumn Property="categoria => categoria.Nome" />
</QuickGrid>
```

O que chega ao browser (simplificado)

```
<table class="quickgrid table">
  <thead><tr><th></th><th>Nome</th></tr></thead>
  <tbody>
    <tr><td><img src="data:..."></td>
        <td>Fruta Fresca</td></tr>
  </tbody></table>
```

Peça	O que faz
`Items`	A consulta (`IQueryable`). A QuickGrid executa-a, com ordenação e página (SQL `OFFSET`/`FETCH`)
`PropertyColumn`	Uma coluna por propriedade. Opções: `Title`, `Sortable`, `Format="N2"`
`TemplateColumn`	HTML à medida em cada linha; `Context` dá nome ao objeto da linha
`Pagination`	`PaginationState` + `<Paginator>`: uma página de cada vez

De onde vem

Pacote NuGet `...Components.QuickGrid` (instalado pelo scaffolding).

`@using` no topo de cada `Index`.

CSS: `quickgrid` vem no pacote; `table` é do Bootstrap.

ALÍNEAS 4.I A 4.K · COMPONENTES

De onde vem cada componente que usamos

Componente	Namespace (@using)	Onde está o @using	Biblioteca
`EditForm`, `InputText`, `InputNumber`, `InputSelect`, `InputCheckbox`, `InputFile`, `ValidationMessage`	`Microsoft.AspNetCore.Components.Forms`	`_Imports.razor`	ASP.NET Core
`NavLink`	`...Components.Routing`	`_Imports.razor`	ASP.NET Core
`AuthorizeView`, `AuthorizeRouteView`	`...Components.Authorization`	`_Imports.razor`	ASP.NET Core
`[Authorize]`	`Microsoft.AspNetCore.Authorization`	topo da página	ASP.NET Core
`QuickGrid`, `PropertyColumn`, `TemplateColumn`, `Paginator`	`...Components.QuickGrid`	topo do `Index`	**NuGet** (scaffolding)
`IDbContextFactory`, `Include`, `FirstOrDefaultAsync`	`Microsoft.EntityFrameworkCore`	topo da página	**NuGet** (EF Core)
`Categoria`, `Produto`, `AppConfig`	`...Entities`, `GestaoLoja`	`_Imports.razor`	O nosso projeto
`IImagensService`, `IMensagensService`; `Aviso`, `Confirmacao`	`GestaoLoja.Interfaces`; `...Components.Shared`	`_Imports.razor` (novo)	O nosso projeto

Tags com maiúscula são componentes (classes C#); tags com minúscula são HTML. As classes CSS (`form-control`, `btn`, `table`, `mb-3`) são do **Bootstrap**, ligado no `App.razor`.

ALÍNEAS 4.I A 4.K · TOPO DA PÁGINA

O topo de cada página: do framework e nosso

ProdutoPages/Create.razor, as diretivas do topo

```
@page "/produtos/create"
@* Do framework: o [Authorize] e o EF Core *@
@using Microsoft.AspNetCore.Authorization
@using Microsoft.EntityFrameworkCore
@* Nosso: as entidades *@
@using GestaoLoja.Entities
@* Quem pode abrir: Admin ou Gestor *@
@attribute [Authorize(Roles = "Admin,Gestor")]
@* Serviços do framework *@
@inject IDbContextFactory<...ApplicationDbContext> DbFactory
@inject NavigationManager NavigationManager
@* Serviços nossos, pela interface (pasta Interfaces) *@
@inject IImagensService Imagens
@inject IMensagensService Mensagens
```

1. O endereço

`@page`: sempre a primeira linha.

2. Os namespaces

`@using`: primeiro os do framework, depois os nossos.

3. Quem pode entrar

`[Authorize(Roles = "Admin,Gestor")]`: só estes roles (tabela `AspNetRoles`). Sem `Roles`, basta ter sessão.

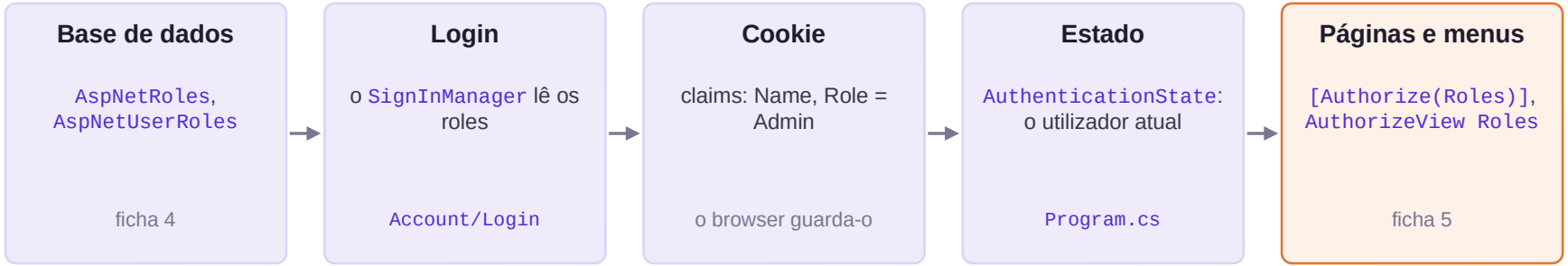
4. Os serviços

`@inject`: primeiro os do framework, depois os nossos, **sempre pela interface**.

O compilador não exige esta ordem: é uma convenção, para se ler o topo de qualquer página da mesma forma. O que já está no `_Imports.razor` vale aqui sem repetir.

CONCEITO · ROLES

Dos roles na base de dados às páginas



AddRoles, na ficha 3

A linha `.AddRoles<IdentityRole>()` do `Program.cs` é o que faz o login ler os roles e pô-los no cookie. Sem ela, `[Authorize(Roles)]` nunca deixava ninguém entrar.

Lidos uma vez, no login

Se se mudar o role de alguém na base de dados, o cookie ainda tem os roles antigos. A pessoa só nota com um novo login (ou quando o Identity renovar o cookie, por omissão ao fim de 30 minutos).

ALÍNEAS 4.I A 4.K · PERMISSÕES

Quem pode fazer o quê na Gestão de Loja

Página ou entrada	Sem sessão	Cliente	Gestor	Admin
Home, Sobre	sim	sim	sim	sim
Categorias, Produtos, Disponibilização: listar, ver, criar, editar, apagar	login	não	sim	sim
Menu: Gerir Encomendas	não	não	sim	sim
Menu: Gerir Utilizadores e Gerir Roles	não	não	não	sim

No topo das 15 páginas CRUD: uma só linha

```
@* Listar, ver, criar, editar e apagar: Admin e Gestor *@
@attribute [Authorize(Roles = "Admin,Gestor")]
@* a vírgula é um OU: basta um dos dois *@
```

Gestor e Admin

O **Gestor gere a loja**. O **Admin** faz o mesmo e **gere as pessoas**: utilizadores e roles (páginas de mais tarde).

ALÍNEAS 4.I A 4.K · AUTHORIZE

[Authorize] protege, AuthorizeView esconde

@attribute [Authorize(Roles = "...")]

No topo da página. **Protege**: a página não abre a quem não tiver o role, mesmo que se escreva o endereço no browser.

<AuthorizeView Roles="..."> ... </AuthorizeView>

Em qualquer sítio da marcação. **Esconde**: o pedaço só é desenhado para quem tiver o role.

Sem o [Authorize]

Qualquer pessoa escrevia `/produtos/create` e entrava, mesmo sem login.

Sem o AuthorizeView

O Gestor via Gerir Utilizadores no menu, clicava, e recebia "Acesso negado".

Os dois juntos: Components/Layout/NavMenu.razor

```
<AuthorizeView Roles="Admin">    @* esconde do Gestor *@
  <NavLink href="Account/GereUser">Gerir Utilizadores</NavLink>
</AuthorizeView>
@* e a página GereUser, quando existir: [Authorize(Roles = "Admin")] *@
```

Nas páginas CRUD

Basta o `[Authorize]`: quem entra pode fazer tudo o que a página mostra.

COMPLEMENTO · ROUTES.RAZOR

Quem barra a entrada, e o que mostra

Como se chega à página	Quem verifica	Sem sessão	Com sessão, sem o role
Ao escrever o endereço, ou F5	O servidor (pedido HTTP)	`/Account/Login?ReturnUrl=...`	`/Account/AccessDenied`
Ao clicar numa ligação da aplicação	O `AuthorizeRouteView`	`<RedirectToLogin />`	template: login outra vez

Components/Routes.razor (o NotAuthorized novo)

```
<NotAuthorized>
  @if (context.User.Identity?.IsAuthenticated == true)
  {  <h1>Acesso negado</h1>  }
  else
  {  <RedirectToLogin />  }
</NotAuthorized>
```

Autenticação (401)

O sistema não sabe quem é: vai fazer login.

Autorização (403)

Sabe quem é, e a resposta é não: login não resolve.

ALÍNEA 4.J · RELAÇÕES

Chaves estrangeiras: InputSelect e Include

ProdutoPages/Create.razor (excerto)

```
<InputSelect @bind-Value="Produto.CategoriaId">
  <option value="0">-- escolha --</option>
  @foreach (var c in categorias)
  {  <option value="@c.Id">@c.Nome</option>  }
</InputSelect>
```

ProdutoPages/Index.razor: a coluna Categoria

```
context.Produtos.Include(p => p.categoria)
// sem Include: produto.categoria vem a null
// com Include: um JOIN com a tabela Categorias
<PropertyColumn Property="p => p.categoria.Nome" />
```

`int CategoriaId` (obrigatória)

Opção inicial com valor 0: a página recusa gravar sem categoria.

`int? ModoEntregaId` (opcional)

Opção vazia (`value=""`): fica `NULL`.

O value é o Id; o texto é o nome

O utilizador escolhe pelo nome, e o `@bind-Value` guarda o `Id` em `CategoriaId`: é a chave estrangeira, a coluna da tabela `Produtos`.

ALÍNEAS 4.I E 4.K · APAGAR

Apagar uma categoria não é como apagar um modo

Categoria (relação obrigatória)

- Na base de dados: `ON DELETE CASCADE` (ficha 4).
- Apagar a categoria faz o SQL Server apagar os seus produtos.
- A página conta-os, avisa e pede confirmação (`<Confirmacao>`).

ModoEntrega (relação opcional)

- Na base de dados: sem cascata.
- O EF Core põe `ModoEntregaId = NULL` nos produtos, **mas só nos que tiver carregado**.
- A página gerada não os carrega: o `DELETE` falhava.

ModoEntregaPages/Delete.razor: carregar os produtos antes de apagar

```
var aApagar = await context.ModosEntrega.Include(m => m.produtos).FirstOrDefaultAsync(m => m.Id == Id);
context.ModosEntrega.Remove(aApagar);   // UPDATE Produtos SET ModoEntregaId = NULL ...; DELETE ModosEntrega
await context.SaveChangesAsync();
```

ETAPA 5 · LOGIN

Entrar como administrador

Data/Inicializacao.cs (ficha 3, alínea 3.g)

```
Email       admin@localhost.com
Password    Is3C..00
```

1. Clicar em Produtos, sem sessão
2. O `Routes.razor` (`RedirectToLogin`) manda para `/Account/Login?returnUrl=...`
3. Login: o `SignInManager` confere o hash e lê os roles
4. Cookie criado; volta a `/produtos`

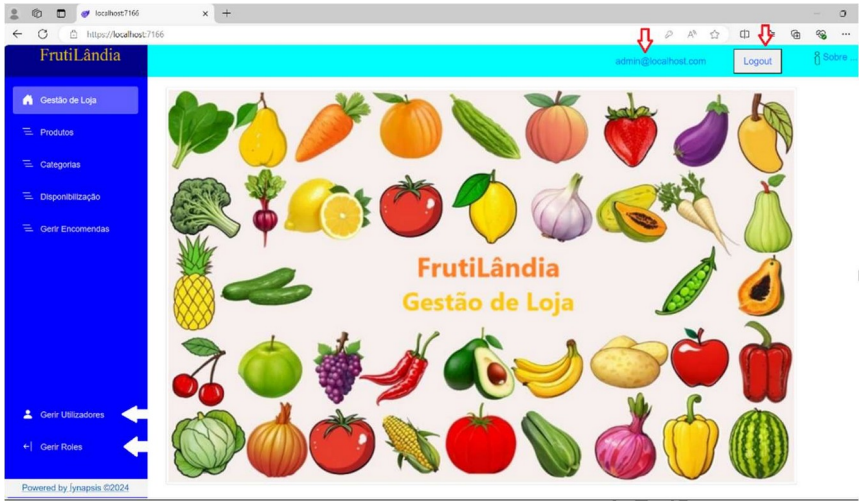
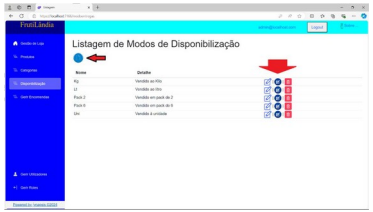


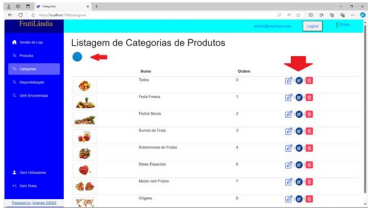
*Figura 15 da ficha, **depois do login como administrador**: em cima, o email e o Logout; no menu, Gerir Encomendas, Gerir Utilizadores e Gerir Roles.*

ETAPA 5 · REGISTOS

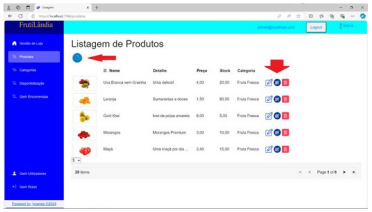
Os dados de exemplo, e as regras a funcionar



Modos



Categorias



Produtos

Experimentar	O que deve acontecer
Gravar um registo válido	Volta à listagem, com a caixa verde ("Produto criado.")
Categoria `legumes`	Caixa vermelha: já existe (o índice não distingue maiúsculas)
Produto com preço `0`	"O preço tem de ser maior do que zero."
Stock `0` e Disponível	"Um produto sem stock não pode estar disponível..."
Imagem `.gif` ou com mais de 500 KB	Mensagem por baixo da caixa; não muda
Apagar uma categoria com produtos	Janela de confirmação; Cancelar não apaga nada

Os dados de exemplo (extra) já lá estão. Inserir à mão: modos, depois categorias, depois produtos (as chaves estrangeiras).

COMPLEMENTO · EXPERIMENTAR OS ROLES

Um segundo utilizador, para ver os roles

1	**Registar**	`gestor@localhost.com`, `Gestor..01`. Entra logo: `RequireConfirmedAccount = false` (extra).
2	**Ver o menu**	Vê o email e o Logout (sem Roles). Não vê Gerir Encomendas (com Roles).
3	**Abrir Produtos**	"Acesso negado": tem sessão, não tem role.
4	**Dar o role Gestor**	Na tabela `AspNetUserRoles`: uma linha com o `Id` do utilizador e o `Id` do role.
5	**Logout e Login**	Gere as três entidades como o Admin. Não vê Gerir Utilizadores nem Gerir Roles.

Atenção: Pendente não faz nada

Quem se regista fica **sem role** e com `Estado = Pendente`. Nenhum código verifica o `Estado`: entra na mesma. Aprovar e dar o role será na página Gerir Utilizadores.

Porquê na tabela

Ainda não há página Gerir Utilizadores (`AddToRoleAsync`).

Porquê o Logout

Os roles estão no cookie, lidos no login: só um login novo traz o role acabado de dar.

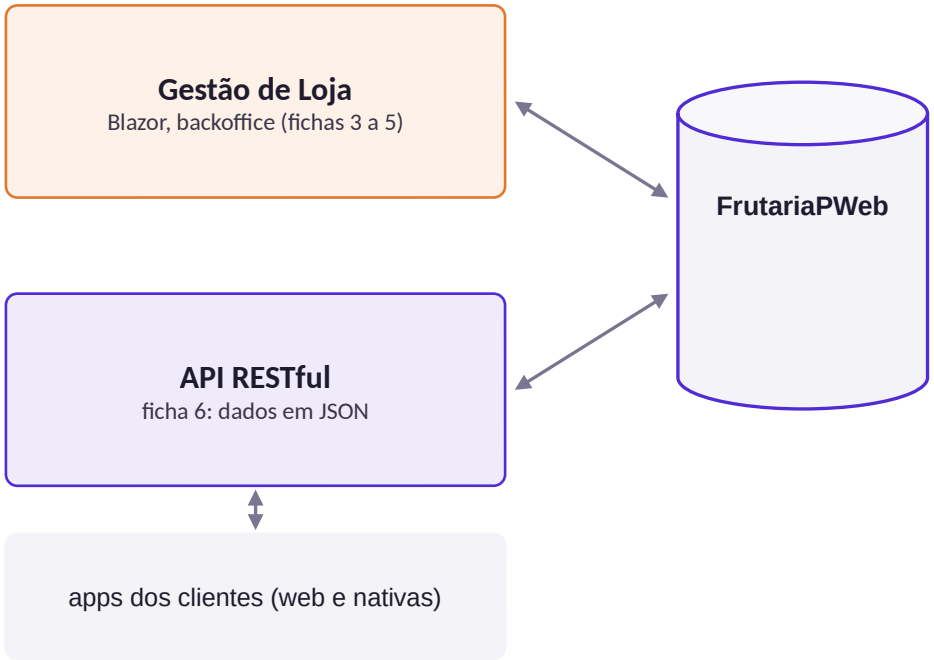
QUANDO ALGO FALHA

Erros frequentes

Sintoma	Causa e solução
Ao arrancar: "Faltam as migrations: ..."	Falta o `Update-Database` (extra, ou depois de um `git pull`)
`Add-Migration`: "IdentityUserLogin requires a primary key"	Falta `base.OnModelCreating(builder)` (extra)
`Update-Database`: "invalid for use as a key column in an index"	`Categoria.Nome` sem `[StringLength(100)]`: é `nvarchar(max)`
Produtos de exemplo sem imagem	A pasta `seed` não estava em `wwwroot/img` no 1.º arranque
Scaffolding falha a compilar, nada é gerado	Ficaram os blocos da `Imagem` do scaffolding anterior (4.e)
CS1503: de `byte[]` para `string`	O `InputText` gerado para `Imagem`: apagar o bloco
"no registered service of type IDbContextFactory"	Falta `AddDbContextFactory` no `Program.cs` (4.h)
"no registered service of type ...IImagensService"	Falta registar o serviço no `Program.cs` (complemento)
As caixas verde e vermelha não aparecem (RZ10012)	Falta `@using GestaoLoja.Components.Shared` no `_Imports`
Apagar um modo: "REFERENCE constraint"	`Delete` sem `Include(m => m.produtos)` (4.k)
Role dado na tabela não tem efeito	Os roles são lidos no login: Logout e Login

A SEGUIR

O que vem a seguir: ficha 6, a API



A mesma base de dados

A API lê os produtos e as categorias que hoje passaram a existir, com as imagens, e entrega-os às aplicações dos clientes.

Voltam a ser úteis

`[JsonIgnore]` (ficha 3); as validações das entidades, que a API aplica sozinha; o `IFormFile`, cujo `Stream` o `IImagensService` já lê; e a tabela `Moradas`.

PARA LEVAR DA AULA

Das classes às páginas protegidas

Migrations e validação

A primeira migration cria, as seguintes alteram. Regras na classe (mensagens) e na base de dados (garantia).

Arquitetura

Browser, servidor Blazor e LocalDB. Frontend e backend correm no servidor; o browser só mostra.

Scaffolding e CRUD

Um campo por coluna; byte[] não compila. Add, Attach, Remove marcam; SaveChangesAsync escreve.

Injeção de dependências

A página pede a interface. Singleton, Scoped, Transient. No Blazor, Scoped dura a ligação: daí a fábrica.

Imagens

IBrowserFile aqui, IFormFile na API: os dois dão um Stream. Os bytes em Imagem, o nome em UrlImagem.

Roles

[Authorize] protege, AuthorizeView esconde. Gestor gere a loja; Admin também as pessoas.